

Prog 4 – pótZH

Bevezetés

A bevezetést és a feladatokat figyelmesen olvassa végig.

A ZH-t zip fájlként kell feltölteni az alábbi formátumban: `név_neptun_kód.zip` (tartalmazza a potzh-feladat és az elkészített potzh-ws-feladat projektet is)

Az alkalmazáserver helye a labor gépein: `C:\apache-tomcat`

Nem forduló kód vagy el nem induló alkalmazás automatikusan 0 pontos ZH-t eredményez.

ChatGPT (vagy bármilyen AI) használata, illetve egymás segítése tilos, ennek észrevétele (akár a ZH során, akár a beadott kód alapján) szintén 0 pontos ZH-t eredményez.

Órai anyag és internet használható a feladatok megoldása során, viszont teljes blokkok másolása esetén az adott feladat 0 pontot ér.

A 0. feladat git kezeléshez fog kapcsolódni, az ott leírtak beugrónak számítanak.

Git hiánya esetén a ZH 0 pontos.

A teljes ZH egyben történő commitálása esetén a ZH szintén 0 pontos.

Több feladat egyben történő commitálása esetén csak az egyik feladat lesz figyelembe véve a javítás során (figyeljen arra, hogy egy feladatot egyben commitálja).

Mivel a ZH jelentős része kapcsolódik az adatbázishoz és az adatok is ott kerülnek tárolásra, így a lentebb megadott Resource segítségével működő adatbázis kapcsolat hiánya esetén a ZH 0 pontot ér.

Ha a docker container létrehozása esetén hibaüzenetet kapna, akkor a docker desktop-on jobb egérgombot nyomva válassza a „Switch to linux containers” lehetőséget, majd a docker beállításai között válassza a „Docker Engine” menüpontot és konfigurációként állítsa be a következőt:

```
{  
  "builder": {  
    "gc": {  
      "defaultKeepStorage": "20GB",  
      "enabled": true  
    }  
  },  
  "experimental": true  
}
```

Ellenőrizze, hogy a `mariadb-java-client-3.1.3.jar` megtalálható-e a tomcat alatt, ha nem akkor a gyakorlaton mutatott módon töltsse le az internetről és helyezze el a megfelelő könyvtárban.

A feladathoz szükség van egy adatbázisra. Amennyiben elindítja a projektben található compose-t, akkor elindul az adatbázis és minden szükséges objektum létrejön benne.

Az adatbázis kapcsolathoz szükséges resource definíciója (ha már van másik resource felvéve azt törölje ki):

```
<Resource name="jdbc/potZH-MariaDB" type="javax.sql.DataSource"
driverClassName="org.mariadb.jdbc.Driver"
factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
url="jdbc:mariadb://localhost:3306/potzh_database" username="potzh_user"
password="PotZHPassword111" maxActive="20" maxIdle="10" maxWait="-1" />
```

Az adatbázisban két felhasználó van.

user1: neki viewer jogosultsága van

user2: neki viewer és admin jogosultsága van

Mind a kettőnek "Password111" a jelszava. A jelszavak Bcrypttel vannak elforgatva.

Git (potzh-feladat és potzh-ws-feladat)

- Hozzon létre egy git repository-t a kapott projektekben.
- Készítsen egy develop branchet, majd ezen a branchen dolgozzon.
- A kapott projekt fájlokat commitálja el egyben.
- Minden feladat megoldását külön-külön commitálja el, figyeljen a beszédes commit message-kre.

A kiadott projekt egy minimális pom.xml-t és az entityket tartalmaz, ezt egészítse ki, hogy a tomcat szerveren web alkalmazásként futtatható legyen.

Az adatbázisban található táblákat és az entityket használva készítse el az alkalmazás repository rétegét, ahol lehet a filmeket lehet menteni, frissíteni, listázni és adott id-val rendelkezőt le lehet kérdezni.

A szerepkörök és a felhasználók esetén szintén készítse el a lekérdezéseket a repository rétegben.

Készítsen egy külön alkalmazást **potzh-ws-feladat** néven, ami szintén futtatható legyen a tomcat alkalmazásszerveren, figyeljen a HTTP és a JMX portszámra!!!!

Az alkalmazásban készítsen el egy web service-t.

Az web service kérésben várjon egy film azonosítót és válaszban adjon vissza egy IMDb értékelést.

Az IMDb értékelés generálásához használja a következő kódrészletet:

```
private final Map<String, Double> map = new HashMap<>();
```

```
public MovieDataResponse getMovieData(MovieDataRequest request) {  
    return new MovieDataResponse(request.getMovieId(), this.map.computeIfAbsent(request.getMovieId(), movieId ->  
        Math.round(ThreadLocalRandom.current().nextDouble(1, 10) * 10 / 10.0));  
}
```

A web service-t tegye elérhetővé a **/ws/movie-imdb** url-pattern-en.

Az elkészített web service-ből generáljon egy WSDL leíró az **/src/main/resources/wsdl** mappába.

Az elkészült leíró használva a potzh-feladat projektben generáljon klienst a web service-hez.

Készítse el az alkalmazásban a film repository-hoz tartozó service-t, minden repository módszernek legyen saját service rétegbeli módszere.

Készítsen a service-ben egy módszert, ami meghívja a kigenerált web service klienst.

Készítsen egy Controller réteget, ami JSON-kkel kommunikál, és üres találat esetén is adjon vissza mindig üzenetet státusszal.

A controller hívja meg a service-ben található metódusokat.

Készítsen egy index oldalt az alkalmazásnak.

Készítsen jsp oldalt, ahol megjelenik a filmek listája.

Az oldal tetején legyen egy page tag, ahol lehet navigálni a listázó oldal és az index oldal között, valamint legyen a tag-en egy kijelentkezés link is.

A listázó oldalon jelenjen meg a filmnek az id-ja, címe, rendezője, megjelenés éve, illetve egy IMDb értékelés oszlopban jelenjen meg az értékelése.

A IMDb értékelés oszlophoz készítsen egy tag library-t, ami az id alapján meghívja a service rétegben elkészített metódusok közül azt, amelyikben a web service kliens hívása történik.

Készítse el az alkalmazás teljes autentikációját.

Működés:

Legyen FORM alapú bejelentkezés.

Az index oldal legyen elérhető bejelentkezés nélkül.

A REST apik legyenek elérhetőek bejelentkezés nélkül.

A listázó oldalt csak „viewer” szerepkörrel legyen elérhető.

A rögzítő oldal csak „admin” joggal legyen elérhető.

Legyen hibás bejelentkezésnek saját oldala.

Korábban a page tag-ben elkészített kijelentkezés linkre kattintva jelentkeztesen ki.